

VIT BHOPAL UNIVERSITY

School of Computer Science and Engineering

PROJECT REPORT

on

Document Processor – A Command-Line Application

(Developed in Python)

Project Title	Document Processor (CLI)
Language / Platform	Python 3 (standard library only, no external dependencies)
Project Type	Command-line application
Submitted By	Aditya Singh
Registration Number	26BAC10006
Program / Branch	B.Tech
Course / Module	CSE1021
Faculty Guide	Ajeet Singh
Institution	VIT Bhopal University
Date	29/09/2026

Table of Contents

Table of Contents	2
1. Introduction	3
2. Problem Statement	3
3. Functional Requirements	4
4. Non-Functional Requirements	4
5. System Architecture	5
6. Design Diagrams	5
7. Design Decisions & Rationale	8
8. Implementation Details	8
9. Screenshots / Results	9
10. Testing Approach	10
11. Challenges Faced	10
12. Learnings & Key Takeaways	10
13. Future Enhancements	11
14. References	11

1. Introduction

Written text is often produced quickly, in the form of lecture notes, chat transcripts, or first drafts, and it commonly contains inconsistent capitalization: sentences that begin in lowercase or words typed in random uppercase. Fixing these errors by hand is tedious, and writers also lack a fast way to see basic facts about a document such as its length, vocabulary size, and most frequent words.

Document Processor is a command-line application written in Python. It reads a plain-text file, normalizes its capitalization so that every sentence starts with a capital letter and the rest of the text is lowercase, produces a statistics report on the console, and saves the corrected text to a new file. The tool uses only the Python standard library, so it runs anywhere Python 3 is installed with no setup beyond cloning the repository.

This report documents the problem, requirements, design, implementation, testing, and lessons learned from building the project.

2. Problem Statement

Users need a quick, repeatable way to (a) correct sentence-level capitalization in a text file and (b) obtain summary statistics about that text, without manually editing the document or using a heavy word processor.

Scope

- **In scope:** reading plain-text files, sentence-case correction, word count, unique vocabulary count, correction count, top-5 frequent words, writing the corrected file, processing several files in one session.
- **Out of scope:** grammar or spelling correction, preserving intentional capitalization (proper nouns, acronyms), non-text formats (.docx, .pdf), and a graphical interface.

Target Users

- Students cleaning up notes and essays
- Writers and editors doing quick formatting passes
- Beginners learning file handling and string processing in Python

3. Functional Requirements

ID	Requirement
FR1	The system shall prompt the user for an input filename and an output filename.
FR2	The system shall read the entire contents of the input text file.
FR3	The system shall convert the text to lowercase and capitalize the first letter of every sentence (after '.', '?' or '!') and of the document.
FR4	The system shall count and display the total number of words.
FR5	The system shall count and display the number of unique words (case-insensitive).
FR6	The system shall count and display the number of characters changed by the correction.
FR7	The system shall display the five most frequent words with their counts.
FR8	The system shall print the corrected text and a formatted report to the console.
FR9	The system shall write the corrected text to the output file.
FR10	The system shall ask whether to process another file and repeat if the answer is 'y' or 'yes'.

4. Non-Functional Requirements

Category	Requirement
Usability	Clear, plain-language prompts and a readable, boxed report layout.
Portability	Runs on Windows, macOS and Linux with Python 3.8+; no external libraries.
Performance	Processes typical documents (up to several MB) in well under a second; time is linear in text size.
Reliability	Handles empty files and files without punctuation without crashing.
Maintainability	Code is split into small functions (read, write, process, main) with a single responsibility each.
Simplicity	Single-file implementation that is easy to read and extend.

5. System Architecture

The application follows a simple layered, single-module design. Input from the keyboard and text file flows through a file I/O layer into a processing layer, and the results leave through the console and an output file.

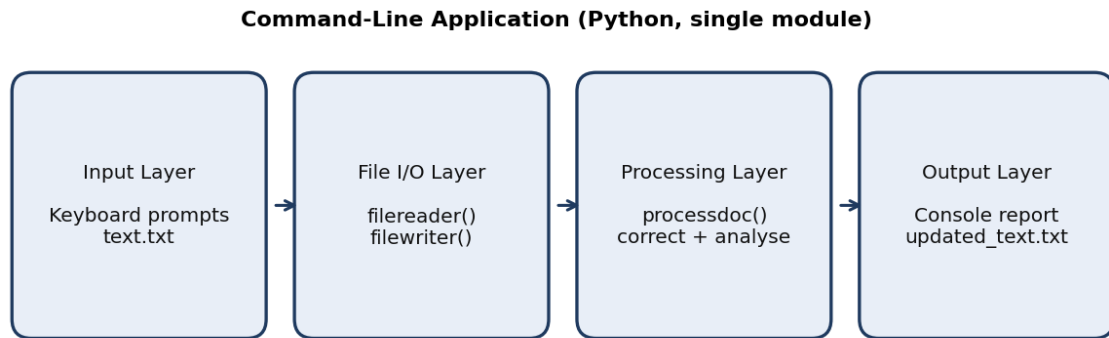


Figure 1: Layered architecture of Document Processor

- **Input layer:** keyboard prompts (filenames, y/n) and the input text file.
- **File I/O layer:** `filereader()` and `filewriter()` isolate all disk access.
- **Processing layer:** `processdoc()` corrects capitalization and computes the statistics.
- **Output layer:** the console report and the corrected output file.

6. Design Diagrams

6.1 Use Case Diagram

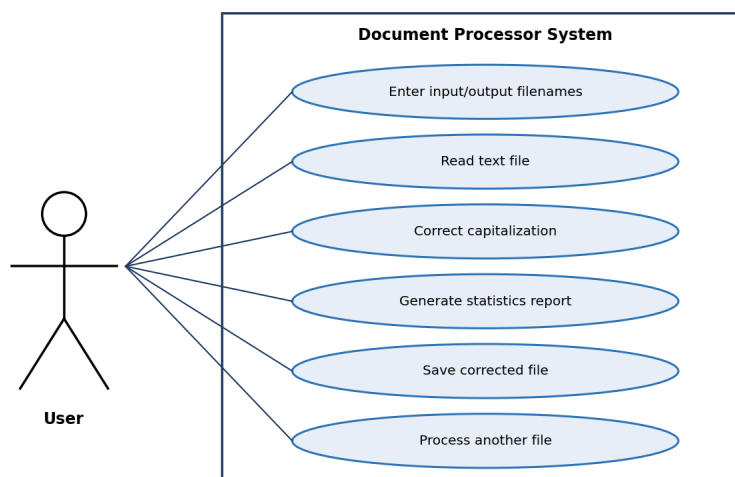


Figure 2: Use case diagram

The only actor is the User, who interacts with all six use cases of the system. There is no administrator or external system.

6.2 Workflow Diagram

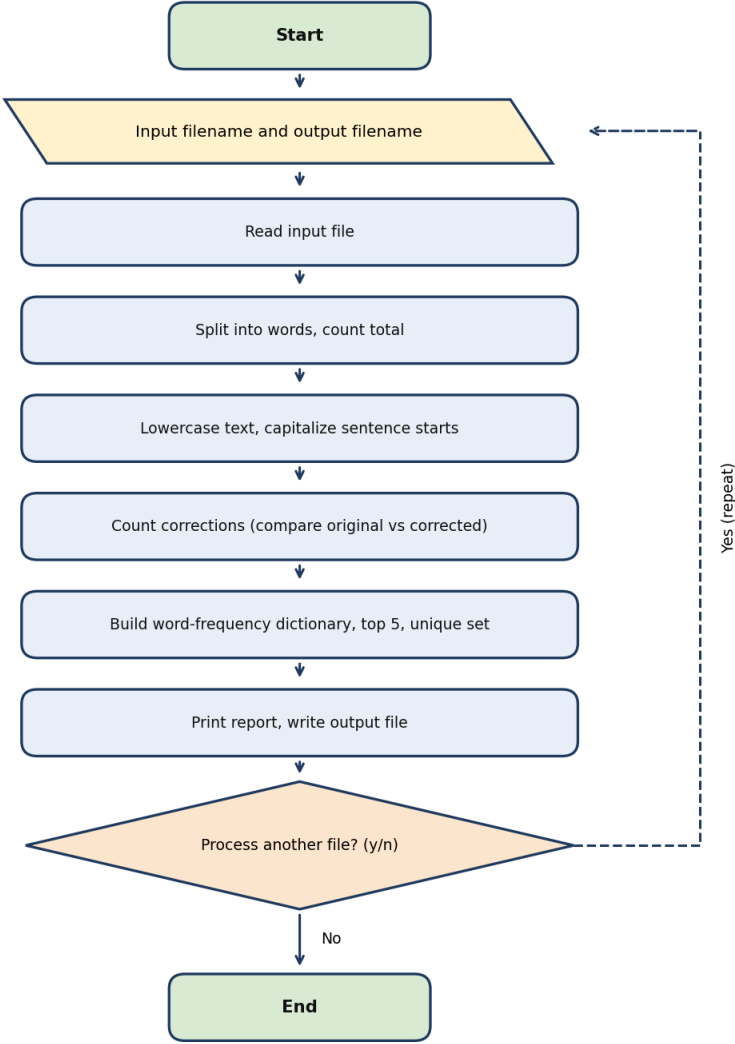


Figure 3: Workflow of a processing run

6.3 Sequence Diagram

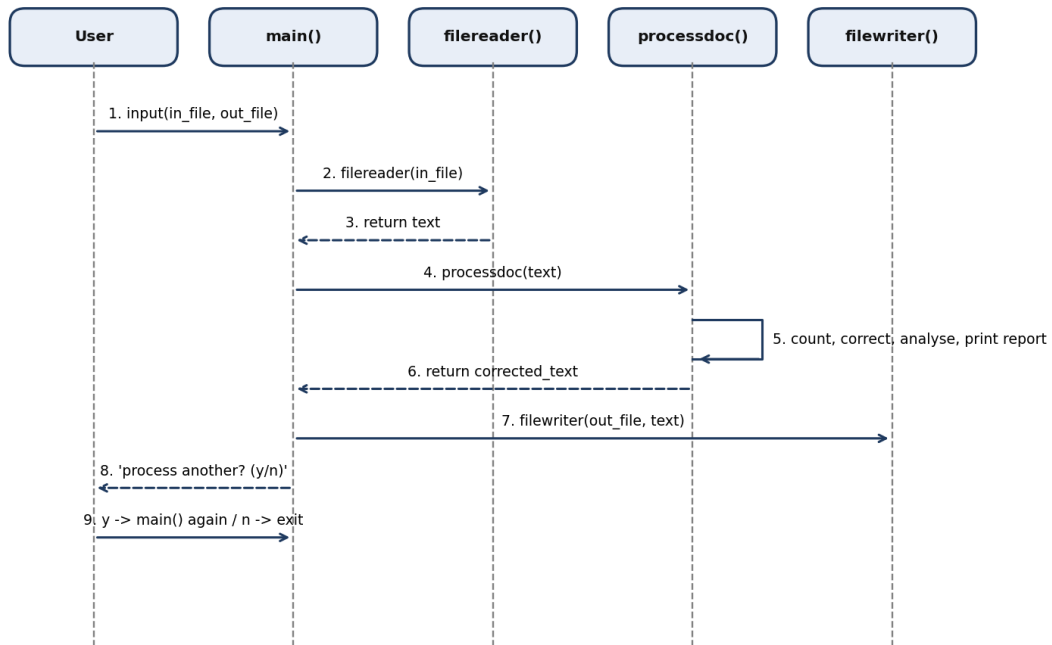


Figure 4: Sequence diagram of one processing run

Steps 1-9 show the order of calls in `main()`: read the file, process the text, write the result, and ask whether to repeat.

6.4 Class / Component Diagram

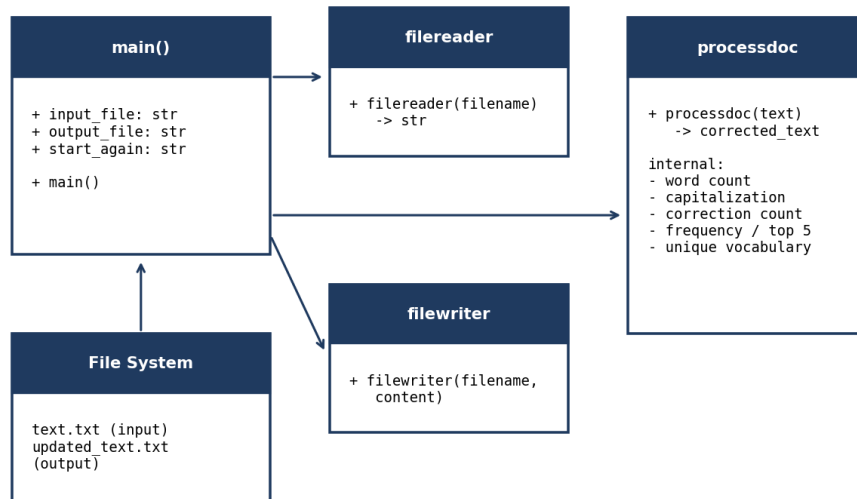


Figure 5: Component diagram

The program uses functions rather than classes, so the components are modules of functions. `main()` coordinates the other three components, which communicate only through function arguments and return values.

6.5 ER Diagram

Not applicable. The application uses no database. Data lives only in memory during a run, and the only persistent storage is the plain-text input and output files, so there are no entities or relationships to model.

7. Design Decisions & Rationale

Decision	Rationale
Python standard library only	Zero installation effort; easy for anyone to run and grade.
Command-line interface	The task is text-in, text-out; a CLI is quicker to build and test than a GUI.
Separate read, write and process functions	Isolates file handling from logic, so the logic can be tested with plain strings.
Lowercase everything, then capitalize sentence starts	A simple rule that gives consistent output regardless of how messy the input is.
Sentence boundaries at '.', '?' and '!'	Covers the common sentence terminators with a single-pass character scan.
Dictionary for word frequency	$O(1)$ average lookups give a linear-time count.
Set for unique vocabulary	Sets remove duplicates automatically after lowercasing.
Correction count = differing characters	Gives an objective measure of how much the text changed.

8. Implementation Details

8.1 Modules and Functions

Function	Purpose
<code>filereader(filename)</code>	Opens the file in read mode, returns its contents as a string.
<code>filewriter(filename, content)</code>	Opens the file in write mode and saves the content.
<code>processdoc(text)</code>	Splits words, corrects capitalization, counts corrections, builds frequency data, prints the report, returns the corrected text.
<code>main()</code>	Prompts for filenames, calls the other functions, and asks whether to process another file.

8.2 Capitalization Algorithm

- Lowercase the whole text.
- Set a flag `capitalize_next = True` at the start.
- Scan character by character: if the flag is set and the character is a letter, uppercase it and clear the flag; otherwise copy it unchanged.
- When the scanned character is '.', '?' or '!', set the flag again.

8.3 Statistics

- **Total words:** `length of text.split()`.

- **Unique vocabulary:** size of a set of lowercased words.
- **Corrections:** number of positions where the original and corrected text differ.
- **Top 5:** a word-to-count dictionary, converted to (count, word) tuples, sorted, and the last five taken.

8.4 Key Code Excerpt

```
for char in lower_text:
    if capitalize_next and char.isalpha():
        correct_char.append(char.upper())
        capitalize_next = False
    else:
        correct_char.append(char)
        if char == '.' or char == '?' or char == '!':
            capitalize_next = True
corrected_text = "".join(correct_char)
```

9. Screenshots / Results

The program was run on the following input file (text.txt):

hello world. this IS a test! do you agree? yes it works. the test is simple.

```
Enter the input filename (e.g., text.txt): text.txt
Enter the output filename (e.g., updated_text.txt): updated_text.txt
=====
DOCUMENT PROCESSING REPORT
=====
Total Words: 16
Unique Vocabulary Size: 15
Total Corrections: 7
Top 5 Words: [('this', 1), ('works.', 1), ('world.', 1), ('yes', 1), ('you', 1)]
Corrected Text: Hello world. This is a test! Do you agree? Yes it works. The test is simple.
=====
Would you like to process another file(y/n): n
Thank you for using
```

Figure 6: Terminal output for the sample input

The output file updated_text.txt contained: Hello world. This is a test! Do you agree? Yes it works. The test is simple.

Why 7 corrections? Five characters changed at sentence starts (h, t, d, y, t become uppercase) and two more changed when 'IS' became 'is'. The report shows 16 total words and 15 unique words, because only 'is' appears twice once case is ignored ('test!' and 'test' still differ).

Observation: the Top 5 list shows every word with a count of 1, even though 'test' and 'is' occur twice. This is because the frequency count is case- and punctuation-sensitive ('test!' vs 'test', 'IS' vs 'is'); see Challenges Faced.

10. Testing Approach

Testing was manual and black-box: the program was run on prepared input files and the console report and output file were compared with the expected results. Each case below was actually executed.

#	Input	Expected	Actual result
1	'hello world. this IS a test! ...' (5 sentences)	Each sentence capitalized, 'IS' to 'is'	Pass: 16 words, 15 unique, 7 corrections
2	Empty file	No crash; zero counts	Pass: 0 words, 0 unique, 0 corrections, empty output
3	'no punctuation here at all'	Only the first letter capitalized	Pass: 'No punctuation here at all', 1 correction
4	'wow!!! really? yes... ok'	Word after each terminator capitalized	Pass: 'Wow!!! Really? Yes... Ok', 4 corrections
5	Non-existent input file	Clear error	Program stops with <code>FileNotFoundError</code> (no friendly message yet)

How to re-run: create the input files, run `python main.py`, enter the filenames, and compare the output. Automated unit tests for `processdoc()` using `unittest` are planned as a future enhancement.

11. Challenges Faced

- **Case- and punctuation-sensitive word counts.** The frequency dictionary treats 'test' and 'test!' as different words and 'IS' and 'is' as different words. In the sample run all top-5 counts were 1 even though 'test' and 'is' occur twice.
- **String methods that appear to do nothing.** Calls such as `start_again.lower()` return a new string and do not change the original, so the result must be assigned.
- **Sentence detection.** Abbreviations ('e.g.', 'Dr.') and decimals are wrongly treated as sentence ends, which capitalizes the next letter.
- **Recursive restart.** Calling `main()` from inside itself works for a few files but grows the call stack on each repeat.
- **Error handling.** A missing input file ends the program with a raw traceback.

12. Learnings & Key Takeaways

- Strings in Python are immutable, so methods like `lower()` and `strip()` return new values.
- Splitting code into small functions makes each part easier to reason about and test.
- Dictionaries and sets give simple, efficient counting and de-duplication.
- Real test runs exposed problems (such as the top-5 counting issue) that reading the code did not.

- Good documentation (README, problem statement, this report) is part of the deliverable, not an afterthought.

13. Future Enhancements

- Normalize words (lowercase, strip punctuation) before frequency counting.
- Replace recursion in `main()` with a `while` loop.
- Use `with open(...)` and `try/except` for safe file handling and friendly error messages.
- Handle abbreviations and preserve proper nouns and acronyms.
- Add automated unit tests and command-line arguments (`argparse`).
- Support `.docx` and `.pdf` input, and provide a simple GUI or web interface.

14. References

- Python Software Foundation. *The Python Standard Library* - built-in types (str, dict, set) and file I/O. <https://docs.python.org/3/library/>
- Python Software Foundation. *The Python Tutorial* - Input and Output. <https://docs.python.org/3/tutorial/inputoutput.html>
- GitHub Docs. *About READMEs*. <https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes>
- Course lecture notes and project guidelines (Project Submission, sections 5 and 6).